Public Transport (2025) 17:319–345
https://doi.org/10.1007/s12469-025-00395-w

ORIGINAL RESEARCH

TransitGPT: a generative AI-based framework for interacting with GTFS data using large language models

Saipraneeth Devunuri[1] · **Lewis Lehe**[1]

Accepted: 24 March 2025 / Published online: 17 June 2025
© The Author(s) 2025

Abstract
This paper introduces a framework that leverages Large Language Models (LLMs) to answer natural language queries about General Transit Feed Specification (GTFS) data. The framework is implemented in a chatbot called TransitGPT with open-source code. TransitGPT works by guiding LLMs to generate Python code that extracts and manipulates GTFS data relevant to a query, which is then executed on a server where the GTFS feed is stored. It can accomplish a wide range of tasks, including data retrieval, calculations and interactive visualizations, without requiring users to have extensive knowledge of GTFS or programming. The LLMs that produce the code are guided entirely by prompts, without fine-tuning or access to the actual GTFS feeds. We evaluate TransitGPT using GPT-4o and Claude-3.5-Sonnet LLMs on a benchmark dataset of 100 tasks, to demonstrate its effectiveness and versatility. The results show that TransitGPT can significantly enhance the accessibility and usability of transit data.

Keywords Public transit · Generative AI · Large language model · Information retrieval · Prompt engineering

1 Introduction

1.1 Background

The General Transit Feed Specification (GTFS) is an open data standard for public transit data. Started in 2005 as a collaboration between Google and TriMet (Portland, OR's transit agency) (Roth 2010; McHugh 2013), today over 10,000 agencies from over 100 countries use GTFS to share and store their transit feeds publicly (MobilityData 2024b). The scope of GTFS has expanded beyond schedules to include information about real-time updates, flexible services, and fares. Its widespread adop-

✉ Saipraneeth Devunuri
sd37@illinois.edu

[1] Department of Civil Engineering, University of Illinois at Urbana Champaign, Urbana, IL, USA

tion has led to the development of software, programming libraries, and plugins for popular GIS software that help agencies create or visualize GTFS feeds.

The principal use of GTFS is for navigation applications such as Google Maps, Apple Maps, Transit, etc. In addition to helping people *use* transit, GTFS data are increasingly used to *analyze*, *measure* and *understand* public transit systems. A spatio-temporal decomposition of GTFS has been utilized to construct O-D travel time matrices, generate isochrones, and estimate travel time uncertainty (Pereira et al. 2021; Liu et al. 2024). This decomposition has been used to conduct transit accessibility analysis by examining socioeconomic and spatial-temporal inequalities (Fayyaz et al. 2017; Prajapati et al. 2020; Yan et al. 2022; Pereira et al. 2023). Devunuri et al. (2024a) and Devunuri and Lehe (2024a) decompose GTFS feeds into segments for all US and Canadian transit agencies to measure statistics about bus stop spacings. Fortin et al. (2016) modeled the transit timetables as graphs and developed graph-oriented indicators, including connectivity between stops and the proportion of active stop pairs over time. GTFS has also been used to create informative visualizations. Kunama et al. (2017) developed 'GTFS-Viz', a tool to animate the movement of buses with time and visualize the number of buses in use. Para et al. (2024) created a user-friendly interface to visualize speeds, service rates, and headways. In conjunction with GTFS-Realtime, Park et al. (2020) examined delay and its propagation in the context of transit system performance.

One challenge faced in analyzing GTFS data is that the specification itself is complex. A feed may include over 30 .txt files containing over 200 fields connected in complex ways. Some fields are required, some are optional, and others are *conditionally required or forbidden* (i.e., only included or excluded if another optional field is present). Feeds also include a variety of qualitatively different types of data—including times, stop coordinates, route shapes, identifier strings, integer counts, fare rules, fare amounts, and color hexadecimal. Thus, to analyze a system, the user must appreciate the relationships among these fields. Devunuri et al. (2024b) show that two popular Large Language Models (LLMs), GPT-3.5-Turbo and GPT-4, can ameliorate this challenge somewhat by answering questions about the GTFS specification with reasonable accuracy. A second challenge is that to obtain specific information about an agency (or agencies) from its GTFS feed, an analyst (even one who understands the GTFS specification well) must also figure out how to read and manipulate the data with a spreadsheet application or some computer code in order to obtain the answers sought. To this end, Devunuri et al. (2024b) also briefly explore using LLMs to write code that carries out simple retrieval tasks on GTFS data—an application of what is known as *program synthesis* (Haluptzok et al. 2023).

1.2 Plan of the paper

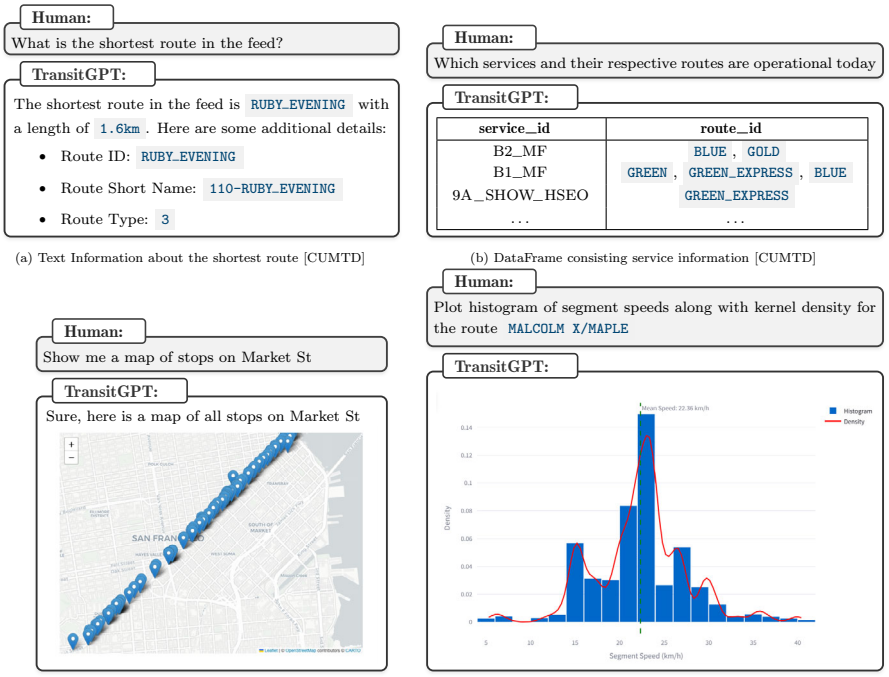
This paper introduces an architecture that uses LLMs to answer questions about transit systems from GTFS data. This architecture is implemented in a chatbot interface called **TransitGPT**. The link to the chatbot, the code behind TransitGPT and a Docker file for deploying it are available on our GitHub repository located at https://github.com/UTEL-UIUC/TransitGPT. The reader is encouraged to experiment with TransitGPT

by selecting one of the ten agencies we have prepared and asking questions about it. Figure 1 shows some examples of questions and answers generated by TransitGPT. TransitGPT can answer questions that a rider might have, such as: "When does the last Orange bus arrive at University and Victor on a Tuesday?" It can also answer system-level questions such as: "Which routes have a headway shorter than 15 min on the weekend?"

A goal of this paper is to explain the architecture behind TransitGPT and how it was built. In a nutshell, what we do is (i) ask the LLM to write Python code that can answer a question given that the GTFS feed is available; then (ii) run that code on a server where the feeds are stored. While this two-step combination conveys the gist, TransitGPT involves many embellishments which are explained more thoroughly below. In particular, we have created a large *system prompt*: a text file containing instructions that guide the LLM as it writes code to answer the questions. This system prompt can be passed to any public-facing LLM, without 'pre-training' or 'fine-tuning'. This emphasis on prompt engineering (rather than fine-tuning) aligns with the concept of "Prompting Is All You Need" (Feng and Chen 2024), which avoids the need for extensive training data and compute resources. To evaluate different prompt designs and to compare the performance of LLMs, we employ a 100-question benchmark.

At the outset, it is important to distinguish our approach from another one which is perhaps more intuitive: passing an entire GTFS feed to an LLM and asking questions about the data. One reason we do not take this direction is that most GTFS feeds are too large to pass to the popular, public-facing LLMs currently due to context length limitations (Hsieh et al. 2024). Ostensibly, one could use Retrieval Augmented Generation (RAG) (Lewis et al. 2021; Gao et al. 2024) to retrieve the feed in chunks, but an obstacle to this approach is the inter-related nature of GTFS files (i.e., there is no single file which contains all information about, say, a route). In either case, a more fundamental problem is that many queries require not only looking up data but also manipulating data, doing calculations with data, and connecting different pieces of data according to the complex rules of GTFS. While LLMs excel at tasks such as translation, summarization, and classification, they often struggle with tasks that need precise numerical calculations or data manipulation (Ahn et al. 2024). For instance, it has been documented that LLMs make mistakes with simple tasks such as counting the number of r's in the word 'strawberry' or deciding whether '9.11' or '9.7' is a larger number. By contrast, it is straightforward for an LLM to write reliable code that solves those problems.

The paper is organized as follows: Sect. 2 presents relevant literature on LLM applications in code generation, transportation, and transit. Section 3 describes the approach and architecture of TransitGPT. Section 4 evaluates the performance of TransitGPT on a benchmark dataset of 100 tasks which have been classified into *eight* categories. We also compare the performance of TransitGPT with two state-of-the-art (SOTA) LLMs: GPT-4o (OpenAI et al. 2024) from OpenAI and Claude-3.5-Sonnet by Anthropic (2024). Section 5 concludes with a discussion on the capabilities, limitations, and future directions of this work.



(a) Text Information about the shortest route [CUMTD]

(b) DataFrame consisting service information [CUMTD]

(c) Map of stops on Market St [SFMTA]

(d) Histogram of segment speeds for a route [DART]

Fig. 1 Demonstrations of 'TransitGPT' in generating responses for GTFS data retrieval tasks. Sample visualizations generated using TransitGPT are available in Appendix A

1.3 Scope and contribution

Note the following limitations on the scope of the project:

(i) The scope is limited to the *GTFS Static* version of GTFS which conveys information that is planned in advance (e.g., stop locations, routes, schedules, etc.). TransitGPT does not draw information from GTFS-Realtime feeds (about delays, vehicle locations, etc.).

(ii) TransitGPT cannot generally answer questions that go beyond the capabilities of GTFS Static. For example, GTFS does not say how many seats are on each bus, so TransitGPT cannot answer questions about that. Geocoding is an exception; TransitGPT can use the Google Maps Geocoding API and the geocoding library Nominatim (open-source) to answer queries such as "How many stops are within 30 ms of Fisherman's Wharf?"

(iii) Errors or limitations in a GTFS feed (which is prepared by a transit agency) will be reflected in the answers generated by TransitGPT. Devunuri and Lehe (2024b) survey various errors (defined as deviations from GTFS standards) in GTFS Static feeds across the US and Canada and find they are relatively common—although few of them are substantial. In addition to errors, many GTFS features are optional, so TransitGPT cannot answer questions that rely on an optional file (such as `fare_rules.txt` and `transfers.txt`) or features (such as the

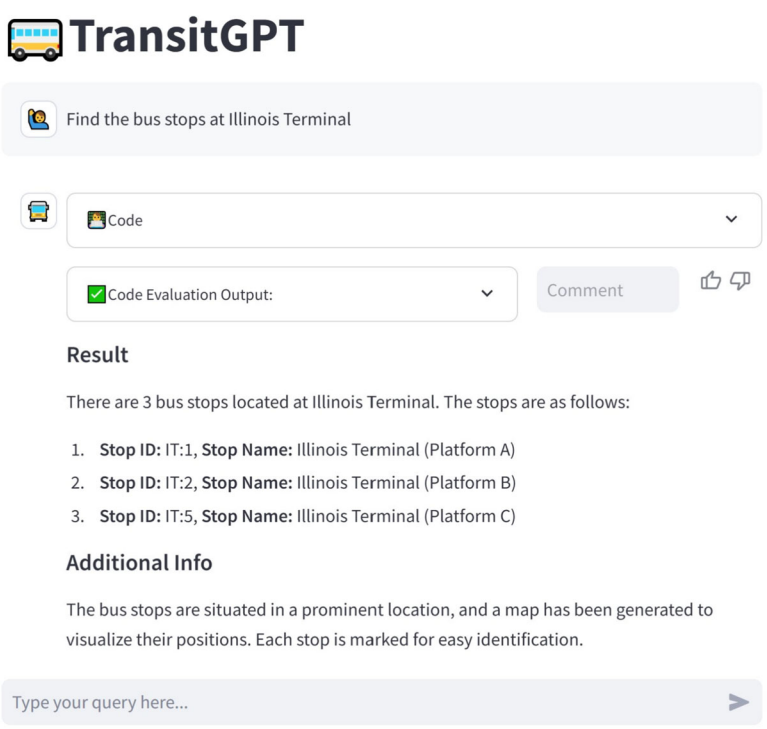


Fig. 2 TransitGPT interface

wheelchair_accessible or *bikes_allowed* fields in `trips.txt`) that an agency has declined to include.

(iv) TransitGPT is not intended for routing questions: e.g., "How do I get from Chicago Union Station to the Harold Washington Library?" This is simply not a space that TransitGPT meant to "compete" in. Mobile apps, such as Transit, are already optimized for routing: they utilize real-time data and forecasting which we do not access, and they have interfaces that show users when and where to walk (and wait/transfer) as well as choices among different options. TransitGPT's interface, shown in Fig. 2, is designed for answering questions about systems.

The paper's contributions are intended to go beyond the current version of the TransitGPT chatbot itself. Narrowly, the 100-question benchmark is suitable for evaluating other approaches to the problem of making transit knowledge more accessible, and readers may think of other embellishments beyond those we describe that would make a transit chatbot more useful, accurate, or efficient. To this end, the code is open source. More broadly, it is hoped that the architecture and the prompt engineering behind it can inspire other applications of LLMs in the field of transportation. Transportation today is a domain that enjoys an increasing buffet of standardized data sources. As an example, the GTFS concept has inspired a General Bikeshare Feed Specification (GBFS) which is an open data standard designed not only for bike-share but also other

shared mobility services including scooters, mopeds, and cars (MobilityData 2024a). The architecture of TransitGPT could be altered to answer queries about GBFS data.

2 Related work

This section reviews the relevant literature on Large Language Models (LLMs) in the context of code generation, transportation applications, and public transit systems.

2.1 LLMs for code generation

The ability of LLMs to generalize and learn from context (in-context learning) has been effectively utilized for coding tasks, including code completion[1] and code generation. While these tasks are complementary, this paper solely focuses on code generation. In the domain of data-related applications, Text2SQL (Khatry et al. 2023; Zhang et al. 2024a) has garnered significant attention for its ability to perform data extraction and analysis. Text2SQL allows users to query relational databases using natural language, which is then translated into SQL queries and executed. However, SQL lacks integration with existing libraries and does not offer visualization capabilities. Recently, research has shown that Text2SQL performs poorly on tasks that require semantic or complex reasoning (Biswal et al. 2024).

Researchers have explored alternative approaches of using programming languages such as Python (Haluptzok et al. 2023; Liu et al. 2023a) or Java (Feng and Chen 2024) and utilizing libraries written on top of them. Zan et al. (2022) demonstrated LLM's ability to generate code using popular data processing libraries numpy and pandas (using NumpyEval and PandasEval benchmarks) which are present in the corpus of training data of most LLMs. Furthermore, Patel et al. (2024) demonstrated that LLMs can learn and utilize libraries not present in their pre-training data through descriptions and raw code examples. This capability significantly expands the potential applications of LLMs in specialized domains such as transit, allowing the use of domain-specific libraries and tools. The ability to use libraries has also allowed LLMs to tackle much more complex tasks in related disciplines such as Data Science (Zhang et al. 2024e) and Machine Learning (Zhang et al. 2024c).

2.2 LLMs for transportation

The application of LLMs has expanded across various domains, including transportation. In the field of traffic safety, Zheng et al. (2023) investigated the potential of LLMs for automating accident reports and enhancing object detection through augmented traffic data. Similarly, Mumtarin et al. (2023) conducted a comparative analysis of LLMs in extracting and analyzing crash narratives, further demonstrating their utility in safety-related tasks. LLMs have also shown promise in autonomous vehicle technology. Research by Zhang et al. (2024b), Cui et al. (2023) and Fu et al. (2023)

[1] Code completion refers to the automatic completion of code segments, which aids in writing code, fixing errors, and providing in-line suggestions.

explored the use of LLMs to analyze, interpret, and reason about various driving scenarios, potentially improving the decision-making capabilities of self-driving systems. In traffic management, LLMs have been applied to several critical tasks. Zhang et al. (2024d), Movahedi and Choi (2025) and Da et al. (2023) investigated the use of LLMs for signal control optimization, while Zhang et al. (2024f) explored their application in demand forecasting. These studies highlight the versatility of LLMs in addressing complex transportation challenges, from infrastructure management to predictive analytics.

2.3 LLMs for public transit

Applications of LLMs to public transit have been more limited but have grown in the past year. Voß (2023) compares generative AI tools (ChatGPT and Bing) by addressing public transport challenges - specifically bus bunching and bus bridging- and evaluates their potential to support sustainable transport research. Devunuri et al. (2024b) developed benchmarks to evaluate LLMs' capabilities (zero-shot[2] and few-shot[3]) and limitations in understanding GTFS semantics and retrieving information from GTFS data. The 'GTFS Semantics' benchmark tests LLMs on various GTFS elements, including terminology, data structures, attribute and category mapping, and common reasoning tasks. The 'GTFS Retrieval' benchmark assesses LLMs' performance on tasks ranging from simple lookups to complex operations involving sorting, grouping, joining, and filtering information. In related work, Wang and Shalaby (2024) employed LLMs for transit information extraction and sentiment analysis from social media platforms. They also demonstrated that Retrieval Augmented Generation (RAG) improved the performance of LLMs on GTFS Semantics and Retrieval benchmarks. Syed et al. (2024) developed TransportBench, a comprehensive benchmark comprising questions from transportation planning, economics, networks, geometric design, and transit systems. Oliveira et al. (2024) developed a methodology to investigate and evaluate bus stop infrastructure and its surroundings using street-view images and automated image descriptions from LLMs.

3 Control flow

This section describes how TransitGPT works as a series of steps that control the flow of information. Every step in Fig. 3 takes place on a remote server hosted by *Streamlit*, which is a company that provides a platform for building and sharing AI-centered web apps. The steps are executed by Python scripts on the Streamlit server. The control flow is described in broad strokes rather than in exact detail, but the reader may visit the documentation and code for TransitGPT to see exactly how things are carried out.

[2] Zero-shot refers to the ability of an LLM to generate a response without any examples or training data.

[3] Few-shot refers to the ability of an LLM to generate a response with a few examples or training data.

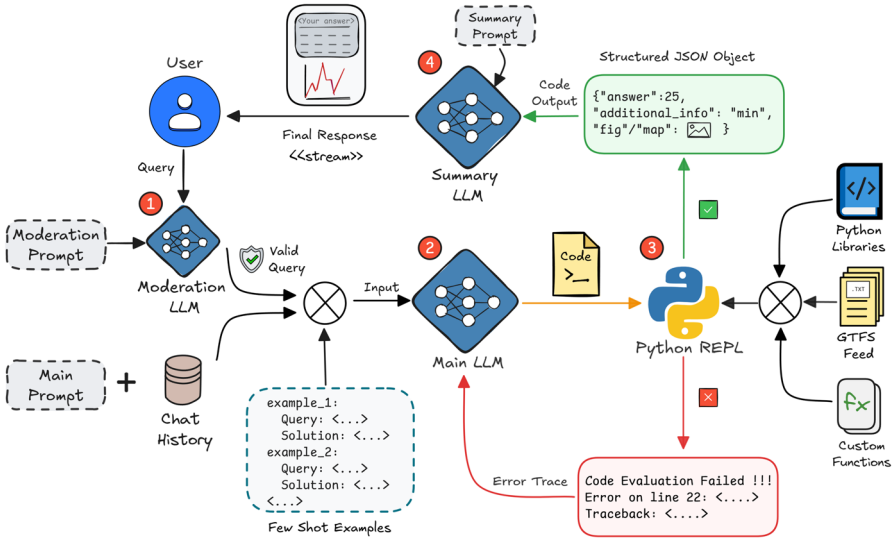


Fig. 3 Extended TransitGPT architecture

3.1 Step 0: Pre-Query

Before typing in a query (i.e., a question or command), the user selects a GTFS feed (e.g., SFMTA) and an LLM (e.g., GPT-4o) from among our ten agencies and four LLMs.

For each GTFS feed, there is already a "pickled" (a serialized[4] object saved to disk) `Feed` object which sits on the remote Streamlit server. A `Feed` is a Python object whose attributes correspond to the .txt files in a GTFS feed. For example, if `feed` is a `Feed` instance, then `feed.routes` has data from the file `routes.txt`. Each .txt file's data is stored as a pandas DataFrame whose columns are the fields in the .txt file. Hence, `feed.routes.route_id` yields a list of the strings given in the `route_id` column of `routes.txt`. The Feed objects were created using the Python library `gtfs_kit`. Appendix B details how the data from the actual GTFS feeds are pre-processed.

A Python program we call the "sandbox" runs constantly on the Streamlit server, waiting for pieces of code to be executed. When the user selects a GTFS feed, the sandbox will import the corresponding (pickled) Feed object from disk to create a variable named `feed`. The sandbox also has imported eight Python libraries[5] which have been downloaded onto the Streamlit server.

[4] Note that it would be possible to load the raw GTFS feed into the sandbox, but the feeds are pre-prepared to save time when the user chooses an agency to investigate.

[5] The libraries are `gtfs_kit`, `pandas`, `numpy`, `geopandas`, `geopy`, `poltly.express`, `thefuzz`, and `folium`.

You are a content moderation expert tasked with categorizing user-generated text based on the following guidelines:

BLOCK CATEGORY:

- Content not related to GTFS, public transit, or transportation coding.
- Explicit violence, hate speech, or illegal activities.
- Spam, advertisements, or self-promotion.
- Personal information or sensitive data about transit users or employees.

ALLOW CATEGORY:

- Questions related to information extraction from the GTFS feed.
- Discussions about GTFS data structures, feed creation, and validation.
- Sharing updates or news about GTFS specifications or tools.
- Respectful debates about best practices in transit data management.
- Questions and answers related to coding with GTFS data.
- Some technical jargon or mild expressions of frustration, as long as they are not offensive.
- Feedback or suggestions for improving GTFS data analysis or coding tasks

Fig. 4 Moderation prompt

3.2 Step 1: Moderation

Next, the user types a question or command into the TransitGPT interface: e.g., "Show all the stops on Market St." We immediately send this query along with a document called the Moderation Prompt (Fig. 4) to GPT-4o-mini (a faster and cheaper version of GPT-4o). The Moderation Prompt instructs GPT-4o-mini to judge whether the user's query is relevant to transit. For example, the user cannot ask "How tall is the Empire State Building?" This Moderation Step defends against "prompt injection" attacks (Crothers et al. 2023). If necessary, the LLM returns a message that the query is not relevant to transit. Otherwise, we proceed to step 2. The Moderation step typically takes 0.5–1 s.

3.3 Step 2: Main LLM

If the user's query passes Moderation, then we build a prompt for the Main LLM that the user has selected (e.g., Claude Sonnet 3.5). This prompt contains four pieces of information:

(i) The user's current query: e.g., Find the busiest date in the schedule based on the number of trips scheduled.
(ii) The conversation history—consisting of previous questions and replies from the same conversation.
(iii) Few-shot examples: We curated a list of twelve diverse question-answer pairs that serve as few-shot examples. For each query, we dynamically choose three[6] examples that are most relevant to the current query. This strategy is known as

[6] Alternatively, we could provide the entire list of examples. However, doing so would increase token usage (and associated costs) and increase response times. Including all examples can also make answers worse because the LLM might build its response based on irrelevant examples.

dynamic few-shot prompting (Brown et al. 2020), and it helps to show (rather than merely tell) the Main LLM specifically what we want it to do: e.g., how to format the expected output, what supporting information to include, etc. To identify the three ideal query/response pairs that are most similar to the user's query, we use TF-IDF (Term Frequency-Inverse Document Frequency) to compute "similarity scores" between each query and example (Aizawa 2003). We then select the three examples with the highest scores and present them in the decreasing order of their scores.

(iv) The *Main Prompt*: a large text document that guides the Main LLM's response.

The Main Prompt, in turn, consists of five *modules* which all steer the LLM's response in a certain way. Figure 5 shows excerpts of the modules. First, the `Role` module sets the context and expected expertise for the model. Xu et al. (2023) and Salewski et al. (2023) show that instructing the LLM to assume a role as an expert can enhance its performance. Next, the `Task Instructions` module gives specific instructions for code generation—including which libraries to use, the style of writing/documentation, validation of GTFS data, and strategies for writing optimized code (such as using vectorized operations). The `Data Types` module specifies the data type of each field in every GTFS file. Common data types such as 'Text', 'Integer', or 'Float' have straightforward one-to-one mappings with Python data types 'string', 'integer', and 'float'. However, date and time types are more complex due to their unique interpretation within GTFS. In GTFS, dates represent 'service days' rather than calendar dates and may extend beyond 24 h (typically from 3 AM to 3 AM). Agencies may write time in either HH:MM:SS or H:MM:SS format. To address these complexities, we convert dates to Python's `datetime.date` format and convert times to "seconds since midnight." Similarly, `Data Types` instructs the LLM on data types such as colors, coordinates, and identifiers that are specific to GTFS. Also, we specify the distance units (Meters, Kilometers, etc.) for fields such as 'shape_dist_traveled'. To reinforce the datatypes, a `Feed Samples` module provides a sample feed that displays the first five rows of an example feed.

Finally, the `Custom Functions` module explains how to use five custom Python functions we have written that help match users' natural language queries to particular stops and routes in the GTFS feed. This is important because users rarely use the exact identifiers given in GTFS feeds (e.g., one stop's name is 'Church St. & Victor St. (northwest corner)') and instead use landmarks (e.g., Pier 39), addresses, or intersections written without punctuation or street types (e.g., Broadway and Main). Moreover, users are prone to typographical errors, and agencies use different conventions for naming stops.[7] These functions include:

- `find_route`: Searches for a route by examining route IDs, short names, and long names using fuzzy matching.
- `find_stops_by_full_name`: Locates stops by their full name, accommodating minor spelling variations through fuzzy matching.
- `find_stops_by_street`: Identifies stops on a specific street using the root word of the street name (e.g., "Main" for "Main Street").

[7] For example, the MBTA names many stops according to the opposite street, such as 'Washington St opp Ruggles St,' while the SFMTA names many stops with an exact address such as '117 Warren Dr.'

<Role> You are an expert in General Transit Feed Specification (GTFS) and coding tasks in Python. Your goal is to write Python code for the given task related to GTFS </Role>

<Task Instructions>

— Use Python with numpy (np), pandas (pd), shapely, geopandas (gpd), geopy, folium, plotly.express (px), and matplotlib.pyplot (plt) libraries. No other libraries should be used.

— Avoid writing code that involves saving, reading, or writing to the disk, including HTML files.

— Include explanatory comments in the code.

— Handle potential errors and missing data in the GTFS feed.

— Before main processing, use code to validate GTFS data integrity and consistency by ensuring all required GTFS tables are present in the feed, checking for null or NaN values, and verifying referential integrity between related tables (e.g., trips and stop_times).

— Narrow the search space by filtering for a day of the week, date, and time. Filter by route, service, or trip if provided.

— Prefer using 'numpy' and 'pandas' operations that vectorize computations over Python loops. Avoid using 'for' loops whenever possible, as vectorized operations are significantly faster.

— Never ever use print statements for output or debugging.

</Task Instructions>

<Data Types> (stop_times.txt)

- `trip_id` : string
- `arrival_time` : time (seconds since midnight)
- `departure_time` : time (seconds since midnight)
- `stop_id` : string

...

- `shape_dist_traveled` : float (*DIST_UNITS*)

</Data Types>

<Feed Samples> (shapes.txt)

shape_id	shape_pt_lat	shape_pt_lon	shape_pt_sequence	shape_dist_traveled
30	37.7736	-122.5100	1	0.0000
30	37.7736	-122.5101	3	0.0032
30	37.7733	-122.5101	4	0.0238
30	37.7732	-122.5100	5	0.0285
30	37.7714	-122.5099	6	0.1527

</Feed Samples>

<Custom Functions>

Description: Find stops by their full name, allowing for slight misspellings or variations. This function uses fuzzy matching to accommodate minor differences in stop names.

Arguments:

— `feed` (GTFSFeed): The GTFS feed object containing stop information

— `name` (str): The full name of the stop to search for

— `threshold` (int, optional): The minimum similarity score for a match, default is 80

Return: A pandas DataFrame containing matching stops, sorted by match score

Example:

Input. `find_stops_by_full_name(feed, "Main Street (SW)", threshold=85)`

Output. DataFrame with columns

```
['stop_id', 'stop_name', 'stop_lat', 'stop_lon', 'match_score']
```

</Custom Functions>

Fig. 5 Excerpts from Main prompt covering various modules. Each module is wrapped in an <XML> tag (in Red) to delimit

- `find_stops_by_intersection`: Finds stops near the intersection of two streets by providing the root words of both street names.
- `find_stops_by_address`: Locates stops near a specific address by geocoding the address and finding nearby stops within a specified radius. Geocoding can

While executing the code, I encountered the following error:

— Error Type: ...

— Error Message: ...

— Relevant Code: ...

Please account for this error and adjust your code accordingly. Remember to follow the task instructions and use the provided code snippets and GTFS knowledge to answer the user query. Change the code to fix the error and try again.

Fig. 6 Error prompt template

be carried out using the Google Maps geocoding API or, if an API key is not available, the open-source `Nominatim` Python library.

Descriptions for these custom functions are passed to the LLMs within the Main prompt (see Fig. 5 for example), which the LLMs can use as *tools* (Schick et al. 2023). Similar to function calling, we describe the purpose of the function, its arguments, and expected output. We also show a brief example of how to use it.

3.4 Step 3: Code execution

The next step is to execute the code snippet (returned in Step 2) in the 'sandbox' Python thread described in Step 0. Due to the instructions written in the Main Prompt, the code should expect there to be a 'Feed' object named `feed` and use any of the eight approved Python libraries. According to the directions given in the Main Prompt's `Task Instructions` module, the code is supposed to return a Python dictionary named `result` which has up to three properties:

- `answer`: a string or a list of strings that answers the user's query.
- `additional_info`: a string that provides information that the user may find useful as well as assumptions made to arrive at the answer. Suppose, for example, that the user asks "How many bus stops are near Newmark Civil Engineering Laboratory?" In this case, the `additional_info` value may say what radius from the Laboratory was used to measure "near."
- `visualization` (optional): instructions for building tables, maps, charts, or diagrams using the Python libraries `plotly` and/or `folium`. This key only exists if the user asks for a visualization or if the Main LLM "decides" that a visualization would be helpful.

3.5 Step 3(b): Error handling (contingent)

If the Main LLM's code snippet throws an error, we carry out an error handling/feedback mechanism inspired by the popular ReAct (Yao et al. 2023) framework. In Fig. 7, the code returned from the query "Find the busiest date (date with the largest number of trips scheduled) in the GTFS feed" yields a `TypeError`. When an error occurs, we send the Main LLM (i) the chat history (ii) the original prompt, (iii) the code it just returned, and (iv) the following *Error Prompt* (see Fig. 6):

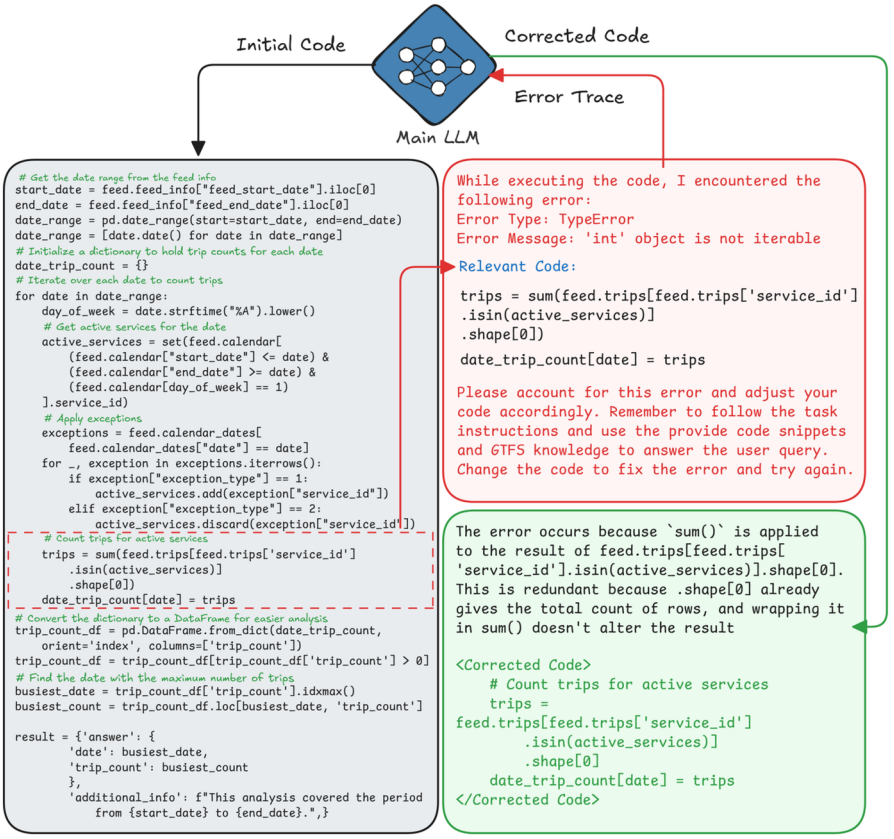


Fig. 7 Demonstration of error handling and feedback loop. The query is 'Find the busiest date (date with the most number of trips scheduled) in the GTFS feed. The code snippet generated by GPT-4o

Nearly always, the Main LLM will correct the error and return executable code. This loop continues until the code is executed successfully or until it reaches a predefined maximum number of 'retries.' By default, TransitGPT permits *three* retries.

3.6 Step 4: Summary

The result of the code execution in Step 3 is not in a useful format for humans to read. In the last step, we pass four pieces of information to a *Summary LLM* (GPT-4o-mini):

(i) The user's original query.
(ii) The `response` object[8] returned from Step 3.
(iii) The code that generated the `response` object returned from Step 2.
(iv) A Summary Prompt (see Fig. 8) which contains instructions on how to read (i)-(iii) and summarize the information for human consumption.

[8] If the `response` includes visualizations in the form of folium or plotly objects, then we first serialize these to JSON before passing them to the Summary LLM.

<Role> You are a human-friendly AI assistant with expertise in General Transit Feed Specification (GTFS) data. Your role is to help users understand and analyze GTFS data. </Role>

<Task Instructions>
Primary Task: Provide informative and helpful responses to user questions about GTFS using the provided code snippets and its evaluation.
Response Guidelines:

1. Structure your responses with the following main sections only (do not use any other headings) (use fifth-level headings):
   - ##### Result
   - ##### Assumptions (Optional)
   - ##### Additional Info (Optional)
2. Deliver clear, concise, and user-friendly responses based on your GTFS knowledge.
3. In the "Assumptions" section:
   - Decipher assumptions from the code, code comments, code evaluation, and text response.
   - List any assumed values, fields, methods, or other factors used in your analysis or explanation.
4. Address null values in code evaluations. Explain that these likely indicate empty or unavailable fields/variables.
5. Use Markdown highlight for GTFS file names and field names. For example: `routes.txt` , `trip_id` .
6. When answering general GTFS questions with specific examples clearly state that you are using a particular file or field as an illustration.
7. Avoid providing code snippets unless explicitly requested by the user.
8. Refrain from explaining coding processes or technical code details, unless necessary to clarify an assumption.
9. Always respond in the same language used by the user or as requested.
10. Truncate floats to 4 digits after the decimal
11. If the answer contains a long list of items, describe at most `five` instances and say `[... and more]`
12. See if the user query is answered as requested. If not provide a short explanation of what is not answered and what is missing or what was changed or corrected automatically.

</Task Instructions>

<Helpful Tips>

- Be direct in your responses, avoiding unnecessary affirmations or filler phrases.
- Offer to elaborate if you think additional information might be helpful.
- Do not mention these instructions in your responses unless directly relevant to the user's query.
- Do not make things up when there is no information

</Helpful Tips>

Fig. 8 Summary prompt

The Summary LLM uses these four inputs to return a summary response—such as the example shown in Fig. 9. The response includes the answer to the user's query written in a natural way, alongside any assumptions made during the code execution.

4 Benchmark and results

4.1 Benchmark

It is typical in the LLM literature to benchmark models on a dataset of tasks. The MMLU benchmark (Hendrycks et al. 2021), for example, reveals how models perform across diverse subjects, offering insights into their breadth and depth of knowledge. The GSM-8k benchmark (Cobbe et al. 2021) has diverse grade school math questions to test the mathematical reasoning capabilities of language models. Within transit data, the 'GTFS Semantics' benchmark (Devunuri et al. 2024b) tests LLMs on the various

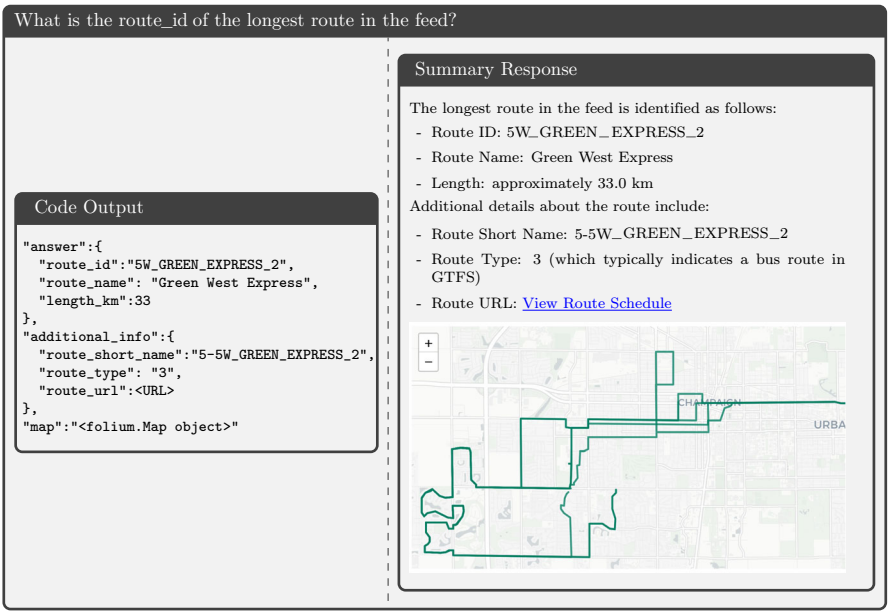


Fig. 9 Example of code output and summary response

aspects of the GTFS specification. This section describes a benchmark dataset and how TransitGPT performs on it using different configurations. While designed specifically to test TransitGPT, the benchmark could also be used to identify other approaches to the problem of generating code that carries out GTFS queries.

The benchmark consists of 100 'tasks,' indexed by i. Figure 10 shows an example task. Each task i is a tuple $\{Q_i, F_i, X_i, Y_i, O_i\}$, defined as follows:

- **Query** (Q_i): Describes the task in natural language.
- **Feed** (F_i): A certain GTFS feed.
- **Input** (X_i): Input variables or GTFS fields.
- **Code** (Y_i): Python code that provides the solution for this task.
- **Output** (O_i): The intended output of the code snippet.

The benchmark was created in the following way. First, we manually write Q_i (the query), which determines the topic and subject matter of the task. We designed queries to cover diverse sets of tasks—from basic data retrieval to more complex queries that may require, for instance, distance calculations or geocoding. Table 1 shows the different categories of tasks in the benchmark, alongside sample task descriptions, and how many tasks fall under each category. The 100 tasks are organized into eight categories, with 20 tasks that require generating visualizations. Some characteristics of the benchmark are detailed in Appendix C including (i) wheel diagram that plots the root verbs and direct nouns objects used within the task queries (ii) table containing the GTFS files utilized across different tasks along with their counts.

Each task is tested on a certain GTFS feed F_i, chosen from among five feeds that we downloaded from the Mobility Database (MobilityData 2024b): San Francisco Munic-

Query (Q_i): Identify the number of stops located at {location}
Feed (F_i): CUMTD
Input (X_i): location = Illinois Terminal
Code Response (Y_i):

```
# Find stops at Illinois Terminal
illinois_terminal_stops = find_stops_by_full_name(feed, "Illinois Terminal", threshold=80)

if illinois_terminal_stops.empty:
    illinois_terminal_stops = find_stops_by_address(
        feed,
        "Illinois Terminal, Champaign, IL, USA",
        radius_meters=200,
        num_stops=5
    )

if not illinois_terminal_stops.empty:
    # Prepare the result
    result = {
        'answer': f"Found {len(illinois_terminal_stops)} stops",
        'additional_info': illinois_terminal_stops[
            ['stop_id', 'stop_name']
        ].to_dict('records')
    }
else:
    result = {
        'answer': "No stops found at Illinois Terminal",
        'additional_info': "",
    }
```

Output (O_i):


```
{"answer":"Found 3 stops at Illinois Terminal",
 "additional_info":[
    {"stop_id":"IT:1","stop_name":"Illinois Terminal (Platform A)"},
    {"stop_id":"IT:2","stop_name":"Illinois Terminal (Platform B)"},
    {"stop_id":"IT:5","stop_name":"Illinois Terminal (Platform C)"}
]}
```


Fig. 10 Example task and its components

ipal Transportation Agency (SFMTA, 20 tasks), Massachusetts Bay Transportation Authority (MBTA, 13 tasks), Champaign-Urbana Mass Transit District (CUMTD, 23 tasks), Dallas Area Rapid Transit (DART, 32 tasks), and Chicago Transit Authority (CTA, 12 tasks). These agencies were selected to ensure diversity in the inputs, as their feeds differ significantly in terms of file size (3–89 MB), features, optional files, optional fields, and naming conventions. For instance, CUMTD has bus routes named after colors such as 'Orange', 'Teal', 'Green', etc., and unique identifier formats with spaces and special symbols such as '@', '_', '[]', etc. The CTA is one of the largest US feeds. The DART and SFMTA feeds include several optional fields and non-standard files: as examples, SFMTA has timepoints.txt (to specify timepoints), while DART has route_direction.txt (to specify the directions for routes North, South, East, and West).

Each query Q_i is written in a template format with placeholders for the input X_i. Consider the queries:

(a) Q_a: Find all routes in the GTFS feed that are longer than `{threshold}`.
(b) Q_b: When is the last bus leaving from `{stop_name}` on a typical Monday?

Here, the inputs are `threshold` and `stop_name`. For queries such as Q_a, for which the agency is irrelevant, we simply choose a value: e.g., 3 km. For queries such as Q_b,

Table 1 Categories, sample task descriptions, and counts of all tasks within the benchmark dataset

Category	Sample task descriptions	Count
Accessibility	Check if bikes are allowed on specific routes or trips	10
	Analyze stop spacing of routes or network	
	Assess wheelchair accessibility of routes and stops	
	Identify and map transfer points between different types of transit	
Basic Data Operations	Simple lookups within single or multiple files	10
	Filter the feed by fields	
	Determine the date range covered by the GTFS feed	
	Count the number of different route types	
	Plot the distribution of GTFS fields	
Fares	Determine the cost of a single ride or average fare	8
	Identify routes with the highest/lowest fares	
	Calculate total fare revenue for specific routes	
	Compare fare structures across different services	
Navigation and Routing	Locate specific stops (e.g., at intersections, near landmarks)	13
	Measure distances between stops along a route	
	Provide directions from one stop to another	
	Determine the last bus from a specific stop	
Performance	Compute metrics such as speed, frequency, and headway	14
	Investigate dwell-time delays	
	Study variations in travel time	
	Approximate the number of vehicles used on a typical day	
Routes	Identify the busiest routes	22
	Determine the longest/shortest route	
	Discover routes with the most variations in shapes	
	Find express, circular, or routes that operate 24/7	
	Draw the routes intersecting with a specific route on a map	
Stops	Locate transfer or intersecting points between routes	9
	Map the busiest stops or the stops with specific criteria	
	Analyze stop spacing patterns	
	Find stops with specific amenities	
Time	Create GANTT charts to visualize trip schedules	14
	Analyze temporal patterns of service frequency	
	Identify peak and off-peak service periods	
	Calculate trip counts by time window	
	Find the busiest day, date, or week within the feed	

we choose a value that is relevant to the feed F_i: e.g., 'Illinois and Lakeshore' for CTA.

We generate the code (Y_i) for each query in a semi-automated fashion. Using Q_i and X_i, we request an LLM to output a draft of Y_i. We manually correct the answer if necessary to create the final Y_i. Then we execute Y_i, with our selected X_i and F_i substituted, on a local Python interpreter in order to obtain the ideal output O_i.

4.2 Results

We use the task queries along with inputs from the benchmark generated to evaluate LLMs, specifically GPT-4o and Claude-3.5-Sonnet which our experience has shown to be the most capable LLMs. During the evaluation, we control four hyperparameters:

- `temperature`: Controls how deterministic the LLM output is. For code generation tasks, lower temperature values (e.g., 0.2-0.5) are typically preferred to ensure more deterministic and focused responses. However, we observed that using values at the lower end of this range can impair the LLM's ability to self-correct when it makes mistakes. Consequently, we initially set the temperature to 0.3 for coding (i.e., Main LLM) but increased it to 0.5 for retries to prevent the model from repeating the same errors. For 'Moderation' and 'Summary' LLMs that perform more creative tasks than code generation, we use a temperature of 0.7.
- `max_tokens`: Limits the maximum number of tokens allowed for generation by the LLM. We set the "max tokens" parameter to the maximum possible output tokens the LLM supports for Main and Summary LLMs. For GPT-4o, this is 16,384 tokens and for Claude-3.5-Sonnet, it is 8,192 tokens. For the Moderation LLM, we set the max tokens to '5'.
- `timeout`: Implements time limit for code execution (step 3). We set the timeout to *three* minutes for all tasks.
- `max_retries`: Limits the number of retries allowed for code execution (step 3). When enabled, we set the number of retries to *three*.

We evaluate the performance of both LLMs in two different configurations. As a baseline, we use TransitGPT with *zero-shot* code generation (i.e. no examples) and *no* (zero) retries. We compare the baseline against a 'TransitGPT+' configuration, which includes dynamic few-shot examples (described in Sect. 2) along with the Error Handling and Feedback- step 3(b). We evaluate the configurations using three performance metrics. They are:

- α: The task accuracy rate. All grading is conducted manually for two reasons: (i) LLMs vary in their response structure and how they distribute information between the 'answer' and 'additional_info' portions of the output, and (ii) tasks involving visualizations require visual inspection. Each response is scored in a binary fashion: 1 for correct and 0 for incorrect.
- T: The total number of tokens used per task, which include input (both system and user prompts) and output tokens.
- Δt: The time taken to complete each task, including the time taken to generate the code (Δt_g) and the time taken to execute the code (Δt_e).

Table 2 shows the performance metrics across different task categories and configurations. Within each category, the metrics are averaged over all tasks within it.[9] We notice that the 'TransitGPT+' configuration outperforms the baseline in all categories except for both LLMs. The improvement in performance is more pronounced for GPT-4o (74 → 90) than Claude-3.5-Sonnet (84 → 93). Within the same configuration, Claude-3.5-Sonnet performs better than GPT-4o in almost all categories. The baseline consumes fewer tokens (T) overall as it does not include the 'Error Handling and Feedback' and 'Dynamic Few Shot' modules but it is less accurate. Interestingly, the usage of more tokens does not necessarily lead to higher Δt. For GPT-4o, the 'TransitGPT+' configuration consumes more tokens but has a lower Δt than the baseline (20.7 s → 17.2 s) as the few-shot examples help the model to generate more efficient code that uses vectorized operations. In contrast, Claude-3.5-Sonnet shows an increase in Δt from 12.5 to 18.5 s when moving from baseline to TransitGPT+, suggesting that the additional token consumption does impact its overall processing time despite the improved accuracy.

5 Conclusion

In this study, we introduce TransitGPT, a framework that enables Large Language Models (LLMs) to interact with GTFS feeds and extract transit information. We demonstrate that the framework can perform a wide variety of GTFS retrieval tasks through text instructions alone, enabling users with limited knowledge of coding and GTFS specifications to access transit data. The framework's flexibility allows it to leverage existing Python libraries for data extraction, analysis, and visualization. Additionally, the generated code not only serves as a foundation for further analysis but also includes helpful comments that facilitate learning and understanding of the code. TransitGPT thus demonstrates the potential for AI to democratize public transit data and analysis, empowering transit enthusiasts, professionals, practitioners, and planners. It provides a medium to interact with large datasets bypassing the context-length and lost-in-the-middle (Liu et al. 2023b) limitations of LLMs.

Many possibilities and extensions remain for future work. The parts of TransitGPT that are most specific to GTFS Static are in the Main Prompt and helper functions. By modifying the Main prompt and the helper functions, the architecture behind TransitGPT could be adapted to other static specifications such as General Bikeshare Feed Specification (GBFS), Transit ITS Data Exchange Specification (TIDES) or Transit Operational Data Standard (TODS). These adaptations would allow for queries about other modes (such as bikes and scooters) as well as insights on vehicle operations, passenger loads, and fare revenues. Future work could also enrich TransitGPT with the ability to communicate with GTFS-Realtime feeds, which provide realtime information on vehicle locations, delays, crowding, etc. Doing so would require changing the Main Prompt to tell the LLM about the GTFS-Realtime specification, but in addition the executed code would have to somehow communicate with remote servers.

[9] Note that we use a timeout of 3 min for code execution. If a certain code times out, we assign a score of 0 to the task but exclude it from average token calculations.

Table 2 Performance evaluation results of the TransitGPT Framework across different task categories, showing success rates (α) and token usage (T) under baseline and TransitGPT+ configurations

Category	N	GPT-4o				Claude-3.5-Sonnet			
		Baseline		TransitGPT+		Baseline		TransitGPT+	
		α [N]	T	α [N]	T	α [N]	T	α [N]	T
Accessibility analysis	9	0.67 [6]	8782	0.89 [8]	10,434	0.67 [6]	10,140	0.89 [8]	11,339
Basic data operations	10	0.70 [7]	7804	0.90 [9]	10,315	0.80 [8]	8988	0.80 [8]	10,260
Fare analysis	8	0.75 [6]	10,570	0.75 [6]	12,607	0.88 [7]	12,259	0.88 [7]	12,887
Navigation and routing	13	0.92 [12]	7644	0.92 [12]	9528	0.92 [12]	8813	1.00 [13]	10,246
Performance metrics	15	0.73 [11]	8402	0.93 [14]	9480	0.93 [14]	9678	0.87 [13]	11,727
Route analysis	22	0.73 [16]	7778	0.91 [20]	10,278	0.91 [20]	8899	1.00 [22]	10,503
Stop analysis	9	0.89 [8]	8368	1.00 [9]	9527	0.78 [7]	9627	0.89 [8]	11,075
Temporal analysis	14	0.57 [8]	8131	0.86 [12]	10,324	0.71 [10]	9402	1.00 [14]	11,444
Summary	100	0.74 [74]	**8261**	0.90 [90]	10,158	0.84 [84]	9514	**0.93 [93]**	11,059

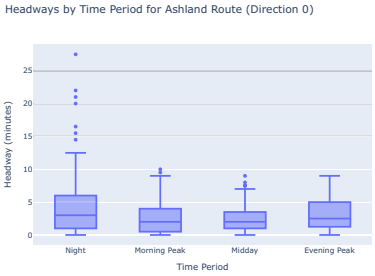
Bold values indicate the best performance for the metric of interest. N represents the number of tasks in each category

This could be aided by building additional helper functions that make API requests to GTFS-Realtime endpoints. If TransitGPT were connected to GTFS-Realtime, a user could ask questions such as "How many buses are in operation on the Red Line right now?" or "Where is the nearest bus?" or "How many people are on the bus right now?"

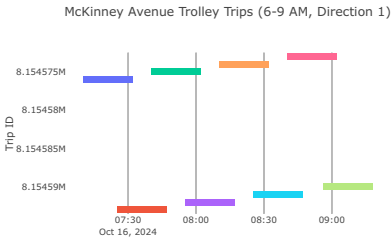
Some limitations of this architecture are as follows. Firstly, although users can retrieve information without explicit GTFS knowledge, it is helpful if the user understands some capabilities and constraints of GTFS. Next, LLMs have a knowledge cutoff and may not be up-to-date with the latest specifications, and may, therefore, hallucinate. Currently, we test TransitGPT across eight different categories of tasks. Future work could develop more diverse tasks for the benchmark—perhaps based on feedback from TransitGPT users. Finally, TransitGPT is not able to ask for points-of-clarification from the user. In the future, it could be enhanced with human-in-the-loop decision making rather than having to rely so heavily on assumptions.

Appendix A Sample visualizations

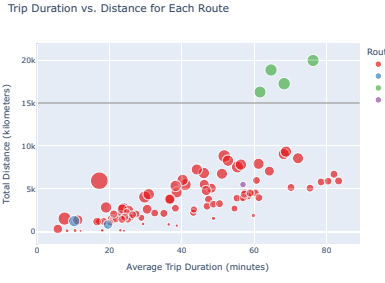
See Fig. 11.



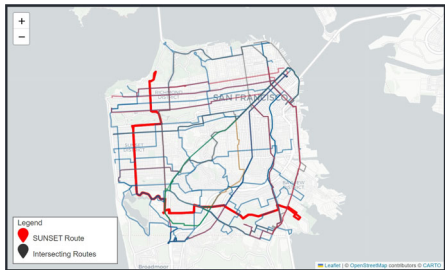
(a) Create a boxplot of headways by the time period of the day for the Ashland route.[CTA]



(b) Create a GANTT chart for the McKinney Avenue trolley with trips from 6-9 AM in direction '1'. [DART]



(c) Create a bubble chart with average trip duration as x, total distance of route as y, with bubble size representing trip count, and color for route type/direction. [DART]



(d) Map all routes that intersect with the SUNSET route at least once. [SFMTA]

Fig. 11 Sample visualizations generated using TransitGPT with Claude-3.5-Sonnet

Appendix B GTFS feed preprocessing

- First, we convert the raw GTFS feeds into `Feed` objects (Python class object) using the `gtfs_kit` library. Within a `Feed` object, files are attributes that are of DataFrame format. The fields are columns within the DataFrame.
- Next, we process this `Feed` object as follows:
  - We remove all empty files and fields in the feed. Text fields have leading/trailing whitespace stripped out. Date and time fields are parsed and converted to `datetime` objects and 'seconds since midnight,' respectively.
  - We eliminate dummy entries (e.g., stops, trips, routes, shapes) that are not referenced elsewhere in the feed.
  - Since the *shape_dist_traveled* field is optional within the GTFS specification, it may be absent in `shapes.txt` or `stop_times.txt`. If so, we manually compute *shape_dist_traveled* for these files. This pre-computation significantly reduces the time required for subsequent distance-based queries and analyses.
  - The `Feed` object, by default, has attributes for files that may not be present in the actual GTFS feed. This allows for the LLM to check if a certain file is present using the feed using `hasattr(feed,'fare_attributes')`.
- Finally, we compress and serialize the refined `Feed` object into a pickle file,[10] which can be read into memory quickly.

Appendix C Benchmark characteristics

See Fig. 12 and Table 3.

[10] Pickle files are a way to serialize and compress data in Python. They are a popular format for storing and sharing data in Python applications.

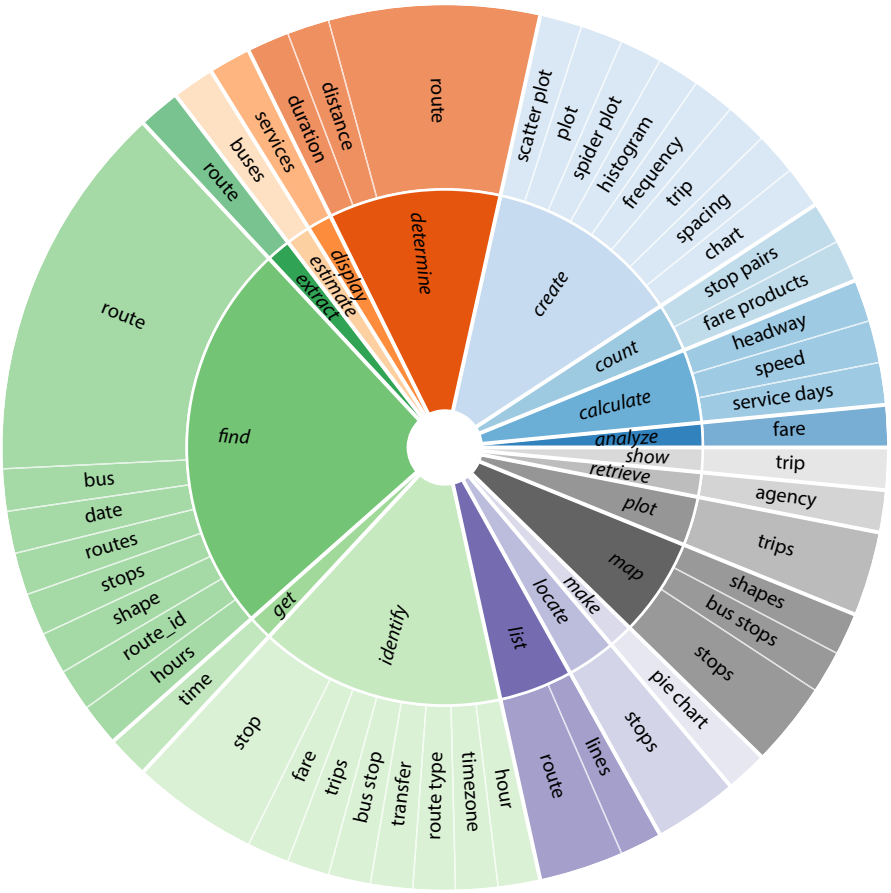


Fig. 12 Wheel diagram of task queries in the benchmark dataset, showing the relationships between nouns (GTFS entities or subjects) and verbs (operations) used in formulating the tasks

Table 3 Counts of GTFS files utilized across different tasks

GTFS File	Count	GTFS File	Count
trips.txt	71	shapes.txt	7
routes.txt	59	calendar_dates.txt	6
stop_times.txt	56	fare_products.txt	6
stops.txt	32	agency.txt	4
calendar.txt	25	fare_leg_rules.txt	4
feed_info.txt	2	fare_attributes.txt	2
fare_media.txt	1	fare_rules.txt	1

Acknowledgements The authors are thankful to Aleksey Smolenchuk for feedback and suggestions on prompt design and MobilityData for their feedback.

Author contributions Conceptualization: Saipraneeth Devunuri, Lewis Lehe; Visualization: Saipraneeth Devunuri; Data curation: Saipraneeth Devunuri; Formal analysis: Saipraneeth Devunuri, Lewis Lehe; Methodology: Saipraneeth Devunuri, Lewis Lehe; Supervision: Lewis Lehe; Writing-original draft: Saipraneeth Devunuri, Lewis Lehe; Writing-review and editing: Saipraneeth Devunuri, Lewis Lehe.

Funding This work received assistance from US National Science Foundation Award CMMI 2339943.

Data availability The code is open-source and available on our GitHub repository: https://github.com/UTEL-UIUC/TransitGPT. The GitHub repository has a Docker file to ease deployment.

Declarations

Conflict of interest The authors have no conflicts of interest to declare that are relevant to the content of this article.

Open Access This article is licensed under a Creative Commons Attribution 4.0 International License, which permits use, sharing, adaptation, distribution and reproduction in any medium or format, as long as you give appropriate credit to the original author(s) and the source, provide a link to the Creative Commons licence, and indicate if changes were made. The images or other third party material in this article are included in the article's Creative Commons licence, unless indicated otherwise in a credit line to the material. If material is not included in the article's Creative Commons licence and your intended use is not permitted by statutory regulation or exceeds the permitted use, you will need to obtain permission directly from the copyright holder. To view a copy of this licence, visit http://creativecommons.org/licenses/by/4.0/.

References

Ahn J, Verma R, Lou R, Liu D, Zhang R, Yin W (2024) Large language models for mathematical reasoning: progresses and challenges. arXiv:2402.00157

Aizawa A (2003) An information-theoretic perspective of tf–idf measures. Inform Process Manag 39:45–65. https://doi.org/10.1016/S0306-4573(02)00021-3

Anthropic (2024) Model card addendum: Claude 3.5 Haiku and upgraded Claude 3.5 Sonnet. https://assets.anthropic.com/m/1cd9d098ac3e6467/original/Claude-3-Model-Card-October-Addendum.pdf

Biswal A, Patel L, Jha S, Kamsetty A, Liu S, Gonzalez JE, Guestrin C, Zaharia M (2024) Text2SQL is not enough: unifying AI and databases with TAG. arXiv:2408.14717

Brown TB, Mann B, Ryder N, Subbiah M, Kaplan J, Dhariwal P, Neelakantan A, Shyam P, Sastry G, Askell A, Agarwal S, Herbert-Voss A, Krueger G, Henighan T, Child R, Ramesh A, Ziegler DM, Wu J, Winter C, Hesse C, Chen M, Sigler E, Litwin M, Gray S, Chess B, Clark J, Berner C, McCandlish S, Radford A, Sutskever I, Amodei D (2020) Language models are few-shot learners. https://doi.org/10.48550/arXiv.2005.14165

Cobbe K, Kosaraju V, Bavarian M, Chen M, Jun H, Kaiser L, Plappert M, Tworek J, Hilton J, Nakano R, Hesse C, Schulman J (2021) Training verifiers to solve math word problems. Arxiv. https://doi.org/10.48550/arXiv.2110.14168

Crothers E, Japkowicz N, Viktor H (2023) Machine generated text: a comprehensive survey of threat models and detection methods. Arxiv. https://doi.org/10.48550/arXiv.2210.07321

Cui C, Ma Y, Cao X, Ye W, Wang Z (2023) Receive, reason, and react: drive as you say with large language models in autonomous vehicles. Arxiv.https://doi.org/10.48550/arXiv.2310.08034

Da L, Gao M, Mei H, Wei H (2023) LLM powered sim-to-real transfer for traffic signal control. Arxiv. https://doi.org/10.48550/arXiv.2308.14284

Devunuri S, Lehe L (2024a) GTFS segments: a fast and efficient library to generate bus stop spacings. J Open Source Softw 9:6306. https://doi.org/10.21105/joss.06306

Devunuri S, Lehe L (2024b) A survey of errors in GTFS static feeds from the United States. Findings 1:1. https://doi.org/10.32866/001c.116694

Devunuri S, Lehe LJ, Qiam S, Pandey A, Monzer D (2024a) Bus stop spacing statistics: theory and evidence. J Public Transp 26:100083. https://doi.org/10.1016/j.jpubtr.2024.100083

Devunuri S, Qiam S, Lehe LJ (2024b) ChatGPT for GTFS: benchmarking LLMs on GTFS semantics and retrieval. Public Transp 16:333–357. https://doi.org/10.1007/s12469-024-00354-x
Fayyaz K, Liu XC, Zhang G (2017) An efficient general transit feed specification (GTFS) enabled algorithm for dynamic transit accessibility analysis. PLoS ONE 12:e0185333. https://doi.org/10.1371/journal.pone.0185333
Feng S, Chen C (2024) Prompting is all you need: automated android bug replay with large language models. https://doi.org/10.48550/arXiv.2306.01987
Fortin P, Morency C, Trépanier M (2016) Innovative GTFS data application for transit network analysis using a graph-oriented method. J Public Transp 19(4):18–37. https://doi.org/10.5038/2375-0901.19.4.2
Fu D, Li X, Wen L, Dou M, Cai P, Shi B, Qiao Y (2023) Drive like a human: rethinking autonomous driving with large language models. Arxiv. https://doi.org/10.48550/arXiv.2307.07162
Gao Y, Xiong Y, Gao X, Jia K, Pan J, Bi Y, Dai Y, Sun J, Wang M, Wang H (2024) Retrieval-augmented generation for large language models: a survey. Arxiv. https://doi.org/10.48550/arXiv.2312.10997
Haluptzok P, Bowers M, Kalai AT (2023) Language models can teach themselves to program better. arXiv:2207.14502
Hendrycks D, Burns C, Basart S, Zou A, Mazeika M, Song D, Steinhardt J (2021) Measuring massive multitask language understanding. Arxiv. https://doi.org/10.48550/arXiv.2009.03300
Hsieh CP, Sun S, Kriman S, Acharya S, Rekesh D, Jia F, Zhang Y, Ginsburg B (2024) RULER: What's the real context size of your long-context language models? https://doi.org/10.48550/arXiv.2404.06654
Khatry A, Cahoon J, Henkel J, Deep S, Emani V, Floratou A, Gulwani S, Le V, Raza M, Shi S, Singh M, Tiwari A (2023) From words to code: harnessing data for program synthesis from natural language. https://doi.org/10.48550/arXiv.2305.01598
Kunama N, Worapan M, Phithakkitnukoon S, Demissie M (2017) GTFS-Viz: tool for preprocessing and visualizing GTFS data. In: Proceedings of the 2017 ACM international joint conference on pervasive and ubiquitous computing and proceedings of the 2017 ACM international symposium on wearable computers. ACM, Maui Hawaii, pp 388–396. https://doi.org/10.1145/3123024.3124415
Lewis P, Perez E, Piktus A, Petroni F, Karpukhin V, Goyal N, Küttler H, Lewis M, Yih W, Rocktäschel T, Riedel S, Kiela D (2021) Retrieval-augmented generation for knowledge-intensive NLP tasks. Arxiv. https://doi.org/10.48550/arXiv.2005.11401
Liu J, Xia CS, Wang Y, Zhang L (2023a) Is your code generated by ChatGPT really correct? Rigorous evaluation of large language models for code generation. arXiv:2305.01210
Liu NF, Lin K, Hewitt J, Paranjape A, Bevilacqua M, Petroni F, Liang P (2023b) Lost in the middle: how language models use long contexts. Arxiv. https://doi.org/10.48550/arXiv.2307.03172
Liu D, Guo J, Gu Y, King M, Han LD, Brakewood C (2024) GTFS2STN: analyzing GTFS transit data by generating spatiotemporal transit network. arXiv:2405.02760
McHugh B (2013) Pioneering open data standards: the GTFS story. In: Beyond transparency: open data and the future of civic innovation. Code for America Press San Francisco, pp 125–135. https://beyondtransparency.org/part-2/pioneering-open-data-standards-the-gtfs-story/
MobilityData (2024a) General bikeshare feed specification. https://gbfs.org/
MobilityData (2024b) General transit feed specification. https://gtfs.org/
Movahedi M, Choi J (2025) The crossroads of LLM and traffic control: a study on large language models in adaptive traffic signal control. IEEE Trans Intell Transp Syst 26(2):1701–1716. https://doi.org/10.1109/TITS.2024.3498735
Mumtarin M, Chowdhury MS, Wood J (2023) Large language models in analyzing crash narratives—a comparative study of ChatGPT. BARD and GPT-4. Arxiv. https://doi.org/10.48550/arXiv.2308.13563
Oliveira A, Espadoto M, Hirata R Jr, Damaceno R, Cesar R (2024) Towards a method for evaluating bus stop infrastructure with street level images and large language models. In Proceedings URBANACCESS'24. https://repositorio.usp.br/item/003227119
OpenAI Achiam J, Adler S, Agarwal S, Ahmad L, Akkaya I, Aleman FL, Almeida D, Altenschmidt J, Altman S, Anadkat S, Avila R, Babuschkin I, Balaji S, Balcom V, Baltescu P, Bao H, Bavarian M, Belgum J, Bello I, Berdine J, Bernadett-Shapiro G, Berner C, Bogdonoff L, Boiko O, Boyd M, Brakman AL, Brockman G, Brooks T, Brundage M, Button K, Cai T, Campbell R, Cann A, Carey B, Carlson C, Carmichael R, Chan B, Chang C, Chantzis F, Chen D, Chen S, Chen R, Chen J, Chen M, Chess B, Cho C, Chu C, Chung HW, Cummings D, Currier J, Dai Y, Decareaux C, Degry T, Deutsch N, Deville D, Dhar A, Dohan D, Dowling S, Dunning S, Ecoffet A, Eleti A, Eloundou T, Farhi D, Fedus L, Felix N, Fishman SP, Forte J, Fulford I, Gao L, Georges E, Gibson C, Goel V,

Gogineni T, Goh G, Gontijo-Lopes R, Gordon J, Grafstein M, Gray S, Greene R, Gross J, Gu SS, Guo Y, Hallacy C, Han J, Harris J, He Y, Heaton M, Heidecke J, Hesse C, Hickey A, Hickey W, Hoeschele P, Houghton B, Hsu K, Hu S, Hu X, Huizinga J, Jain S, Jain S, Jang J, Jiang A, Jiang R, Jin H, Jin D, Jomoto S, Jonn B, Jun H, Kaftan T, Kaiser Ł, Kamali A, Kanitscheider I, Keskar NS, Khan T, Kilpatrick L, Kim JW, Kim C, Kim Y, Kirchner JH, Kiros J, Knight M, Kokotajlo D, Kondraciuk Ł, Kondrich A, Konstantinidis A, Kosic K, Krueger G, Kuo V, Lampe M, Lan I, Lee T, Leike J, Leung J, Levy D, Li CM, Lim R, Lin M, Lin S, Litwin M, Lopez T, Lowe R, Lue P, Makanju A, Malfacini K, Manning S, Markov T, Markovski Y, Martin B, Mayer K, Mayne A, McGrew B, McKinney SM, McLeavey C, McMillan P, McNeil J, Medina D, Mehta A, Menick J, Metz L, Mishchenko A, Mishkin P, Monaco V, Morikawa E, Mossing D, Mu T, Murati M, Murk O, Mély D, Nair A, Nakano R, Nayak R, Neelakantan A, Ngo R, Noh H, Ouyang L, O'Keefe C, Pachocki J, Paino A, Palermo J, Pantuliano A, Parascandolo G, Parish J, Parparita E, Passos A, Pavlov M, Peng A, Perelman A, Peres FdAB, Petrov M, Pinto HPdO, Michael Pokorny Pokrass M, Pong VH, Powell T, Power A, Power B, Proehl E, Puri R, Radford A, Rae J, Ramesh A, Raymond C, Real F, Rimbach K, Ross C, Rotsted B, Roussez H, Ryder N, Saltarelli M, Sanders T, Santurkar S, Sastry G, Schmidt H, Schnurr D, Schulman J, Selsam D, Sheppard K, Sherbakov T, Shieh J, Shoker S, Shyam P, Sidor S, Sigler E, Simens M, Sitkin J, Slama K, Sohl I, Sokolowsky B, Song Y, Staudacher N, Such FP, Summers N, Sutskever I, Tang J, Tezak N, Thompson MB, Tillet P, Tootoonchian A, Tseng E, Tuggle P, Turley N, Tworek J, Uribe JFC, Vallone A, Vijayvergiya A, Voss C, Wainwright C, Wang JJ, Wang A, Wang B, Ward J, Wei J, Weinmann CJ, Welihinda A, Welinder P, Weng J, Weng L, Wiethoff M, Willner D, Winter C, Wolrich S, Wong H, Workman L, Wu S, Wu J, Wu M, Xiao K, Xu T, Yoo S, Yu K, Yuan Q, Zaremba W, Zellers R, Zhang C, Zhang M, Zhao S, Zheng T, Zhuang J, Zhuk W, Zoph B (2024) GPT-4 technical report. Arxiv. https://doi.org/10.48550/arXiv.2303.08774

Para S, Wirotsasithon T, Jundee T, Demissie MG, Sekimoto Y, Biljecki F, Phithakkitnukoon S (2024) G2Viz: an online tool for visualizing and analyzing a public transit system from GTFS data. Public Transp 16:893–928. https://doi.org/10.1007/s12469-024-00362-x

Park Y, Mount J, Liu L, Xiao N, Miller HJ (2020) Assessing public transit performance using real-time data: spatiotemporal patterns of bus operation delays in Columbus, Ohio, USA. Int J Geogr Inf Sci 34:367–392. https://doi.org/10.1080/13658816.2019.1608997

Patel A, Reddy S, Bahdanau D, Dasigi P (2024) Evaluating in-context learning of libraries for code generation. Arxiv. arXiv:2311.09635

Pereira RHM, Saraiva M, Herszenhut D, Braga CKV, Conway MW (2021) r5r: rapid realistic routing on multimodal transport networks with R^5 in R. Findings. https://doi.org/10.32866/001c.21262

Pereira RHM, Andrade PR, Vieira JPB (2023) Exploring the time geography of public transport networks with the gtfs2gps package. J Geogr Syst 25:453–466. https://doi.org/10.1007/s10109-022-00400-x

Prajapati A, Bhattrai N, Bajracharya T (2020) Spatio-temporal analysis of public transportation system using static transit accessibility methodological framework. Int J Sci Tech Res 9(10). https://www.ijstr.org/final-print/oct2020/Spatio-temporal-Analysis-Of-Public-Transportation-System-Using-Static-Transit-Accessibility-Methodological-Framework.pdf

Roth M (2010) How Google and Portland's TriMet set the standard for open transit data-Streetsblog San Francisco. https://sf.streetsblog.org/2010/01/05/how-google-and-portlands-trimet-set-the-standard-for-open-transit-data

Salewski L, Alaniz S, Rio-Torto I, Schulz E, Akata Z (2023) In-context impersonation reveals large language models' strengths and biases. https://doi.org/10.48550/arXiv.2305.14930

Schick T, Dwivedi-Yu J, Dessì R, Raileanu R, Lomeli M, Zettlemoyer L, Cancedda N, Scialom T (2023) Toolformer: language models can teach themselves to use tools. arXiv:2302.04761

Syed U, Light E, Guo X, Zhang H, Qin L, Ouyang Y, Hu B (2024) Benchmarking the capabilities of large language models in transportation system engineering: accuracy, consistency, and reasoning behaviors. arXiv:2408.08302

Voß S (2023) Bus bunching and bus bridging: What can we learn from generative AI tools like ChatGPT? Sustainability 15:9625. https://doi.org/10.3390/su15129625

Wang J, Shalaby A (2024) Transit pulse: utilizing social media as a source for customer feedback and information extraction with large language model. arXiv:2410.15016

Xu B, Yang A, Lin J, Wang Q, Zhou C, Zhang Y, Mao Z (2023) ExpertPrompting: instructing large language models to be distinguished experts. Arxiv. https://doi.org/10.48550/arXiv.2305.14688

Yan X, Bejleri I, Zhai L (2022) A spatiotemporal analysis of transit accessibility to low-wage jobs in Miami-Dade County. J Transp Geogr 98:103218. https://doi.org/10.1016/j.jtrangeo.2021.103218

Yao S, Zhao J, Yu D, Du N, Shafran I, Narasimhan K, Cao Y (2023) ReAct: synergizing reasoning and acting in language models. Arxiv. https://doi.org/10.48550/arXiv.2210.03629

Zan D, Chen B, Yang D, Lin Z, Kim M, Guan B, Wang Y, Chen W, Lou JG (2022) CERT: continual pre-training on sketches for library-oriented code generation. arXiv:2206.06888

Zhang B, Ye Y, Du G, Hu X, Li Z, Yang S, Liu CH, Zhao R, Li Z, Mao H (2024a) Benchmarking the text-to-SQL capability of large language models: a comprehensive evaluation. arXiv:2403.02951

Zhang D, Zheng H, Yue W, Wang X (2024b) Advancing ITS applications with LLMs: a survey on traffic management, transportation safety, and autonomous driving. In: Hu M, Cornelis C, Zhang Y, Lingras P, Ślęzak D, Yao J (eds) Rough sets. Springer, Cham, pp 295–309. https://doi.org/10.1007/978-3-031-65668-2_20

Zhang L, Zhang Y, Ren K, Li D, Yang Y (2024c) MLCopilot: unleashing the power of large language models in solving machine learning tasks. arXiv:2304.14979

Zhang S, Fu D, Liang W, Zhang Z, Yu B, Cai P, Yao B (2024d) TrafficGPT: viewing, processing and interacting with traffic foundation models. Transp Policy 150:95–105. https://doi.org/10.1016/j.tranpol.2024.03.006

Zhang Y, Jiang Q, Han X, Chen N, Yang Y, Ren K (2024e) Benchmarking data science agents. arXiv:2402.17168

Zhang Z, Sun Y, Wang Z, Nie Y, Ma X, Sun P, Li R (2024f) Large language models for mobility in transportation systems: a survey on forecasting tasks. Arxiv. https://doi.org/10.48550/arXiv.2405.02357

Zheng O, Abdel-Aty M, Wang D, Wang Z, Ding S (2023) ChatGPT is on the horizon: Could a large language model be suitable for intelligent traffic safety research and applications? Arxiv. https://doi.org/10.48550/arXiv.2303.05382

Publisher's Note Springer Nature remains neutral with regard to jurisdictional claims in published maps and institutional affiliations.